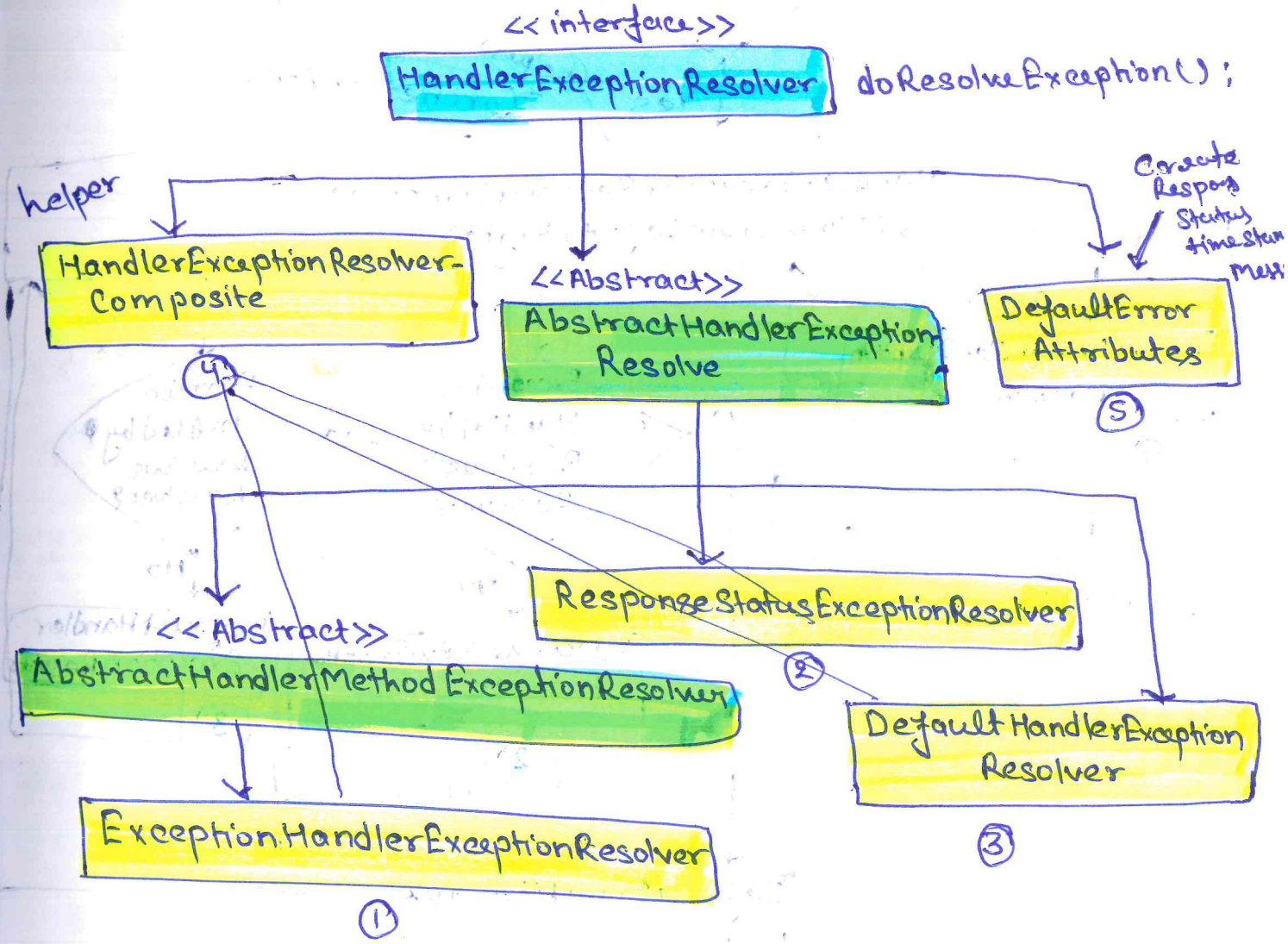
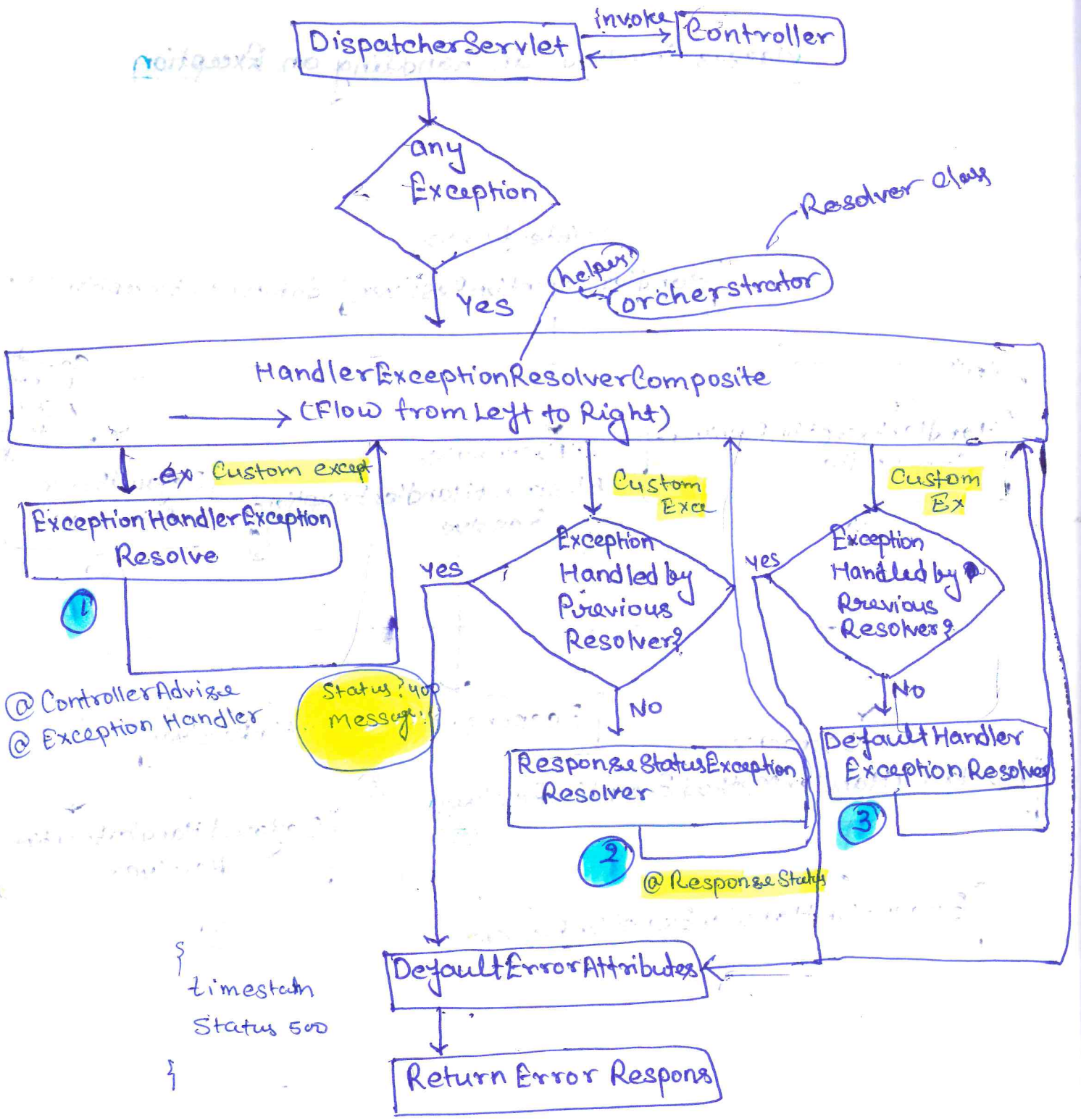
Leacture 21 — Exception-Handling in Spring Boot — 1

Classes involved in handling an Exception



Concept && Coding

Sequence of when any exception occurs:



* HandlerExceptionResolverComposite class invoke this class in sequence.

Concept && Coding

Flow of the Exception

```
@RestController
@RequestMapping(value = "/api/")
Public class UserController{

    @GetMapping(path = "/get-user")
    Public String getUser(){
        throw new NullPointerException("throwing null pointer exceptior
                                        for testing");
    }
}
```

Output:

```
"timestamp":
"Status": 500
"error" Internal Server Error".
"path": "/api/get-user"
```

How ?

```
@RestController
@RequestMapping(value = "/api/")

Public String getEmployee(){
    Throw new CustomException(HTTPStatus.BAD_REQUEST,
                              "request is not correct, UserID is
                                missing");
}
}
```

Concept && Coding

Qu No 1:- Why for both output is same?

Qu No 2:- Why my Return Code "BAD_REQUEST" i.e 400 and my error message is not shown in Output?

Ans:- In the both example, we are not creating the ResponseEntity object, we are just returning the exception, some other class is creating the ResponseEntity Object.

* none of the resolver to handle your exception and than DefaultErrorAttributes by default value it will set 500.

But

* If we want full Control and don't want to rely on Exception Resolvers, then we have to create the ResponseEntity Object.

```
@GetMapping(path = "/get-user")
Public ResponseEntity<?> getUser(){
    try{
        throw new CustomException(HttpStatus.BADREQUEST, "UserID is
                                                          missing");
    }
    catch(CustomException e){
        ErrorRespononse errorrespons =new ErrorResponse(new Date(), e.get-
                                                          Message(), e.getStatus().value())
    return new ResponseEntity<>(errorresponse, e.getStatus());
    }
    catch(Exception e){
        ErrorResponse errorResponse = new Response (new Date(), e.getMess
    return new ResponseEntity<>(, HttpStatus.INTERNAL-SERVER
```

Concept && Coding

So, when we don't return ResponseEntity, we have simply throw.

then in Exception scenario, exception passes through each Resolver like "ExceptionHandlerExceptionResolver", "ResponseStatusExceptionResolver" and "DefaultHandlerExceptionResolver" in sequence

* Each Resolver set the proper status and message in HTTP response for exceptions which they are taking care of.

and

NullPointerException and CustomException all the 3 Resolvers, do not understand, so status and message is not set.

So, when control reaches to "DefaultErrorAttributes" class, it fills the values in HTTP Respons with default values.

Concept && Coding

So Now question is : What exception does "ExceptionHandlerExceptionResolver", "ResponseStatusExceptionResolver" and DefaltHandlerExceptionResolver handle ?

1 ExceptionHandlerExceptionResolver

Responsible for handling below annotations:

- @ExceptionHandler
- @ControllerAdvice

Controller level Exception handling:

```
@RestController
@RequestMapping(value = "/api/")

    @GetMapping(path = "/get-user")
    Public ResponseEntity<?> getUser(){
        throw new CustomException(HttpStatus.BAD_REQUEST,
                                  "UserID is missing");
    }

    @ExceptionHandler(CustomException.class)
    Public ResponseEntity<String> handleCustomException(
                                  CustomException ex){
        return new ResponseEntity<>(ex.getMessage(),
                                    ex.getStatus());
    }

}
```

Output

1 UserID is missing 400 Bad Request

Concept && Coding

Another Use Case 1 : Show to returning Error Response object instead of just message:

Use Case 2 :- To show multiple @Exception-Handler in single Controller class:

and also we can do this

```
@ExceptionHandler({CustomException.class, IllegalArgumentException.class})
handleCustomException(Exception ex).
```

Use Case 3 :- @ExceptionHandler not returning ResponseEntity and let "DefaultErrorAttributes" to create the ResponseEntity.

```
@ExceptionHandler(CustomException.class)
public void handleCustomException(HttpServletResponse response,
                                  CustomException ex) throws
                                  IOException {
response.sendError(HttpStatus.BAD_REQUEST.value,
                   ex.getMessage());
}
}
```

HttpsResponse
HttpserRequest
Exception.

Concept && Coding

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
Public class CustomException extends RuntimeExceptions{

    CustomException(String message){
        super(message);
    }
}
```

Use case 2:- If we want to give message.

```
@ResponseStatus(value = HttpStatus.BAD_REQUEST, reason = "Invalid
                                                Request Passed")
Public class CustomException extends RuntimeException{
    HttpStatus status

    CustomException(HttpStatus status, String message){
        super(message);
        this.status = status;
    }
}
```

Output :-

```
timestamp
"Status": 400,
"error": "Bad Request"
"message:- "Invalid Request Sent",
"path": "/api/get-user)
```

ExceptionHandlerResolver
S = Forbidden
M = .

R.S.E.R
S- Bad Rea
mesasage = Invalid

* Resolver is not handling. ServletInvocableHandlerMethod is handle this. this is a spring framework.

Concept && Coding

3. DefaultHandlerExceptionResolver

Handle Spring Framework related exceptions only like MethodNoFound, NoResourceFound etc.

Concept && Coding